ECON 3420A Assignment 1: Build Your Own Portfolio

Lecture Note Part 1-3 | 100 marks

Replace with your name and student ID

2026-10-16

How to use this template:

- Rename it `studentID_assignment_one.qmd` and keep it in the same folder as `portfolio_model.py`, `risk_aversion_quiz.py` and the `data` folder.
- Replace every `[WRITE …]` and `[INSERT …]` placeholder with your answer.
- Cells marked `#| eval: false` are switched off. Complete the matching function in `portfolio_model.py`, then delete that line to switch the cell on.
- Work in this order: Questions 1–4 → render (this saves your portfolio for Question 5) → run the Question 5 quiz in a terminal → switch on Questions 5–6 → render the final PDF.
- Rendering only reads local files. It never downloads data or asks for keyboard input.

(a) My beta prediction (for Q2)

Record this **before** you see any regression results.

Prediction	My answer and reason	What I later observed
Which stock will have the highest beta?	MSTR. Its monthly volatility is about 31%, roughly 7 times SPY's 4.5% and far above AMD (20%), NVDA (14.5%), META (12.4%) and LLY (9.1%). Beta scales with a stock's volatility relative to the market's, and MSTR's price is driven by bitcoin and leverage, which should amplify market risk-on and risk-off moves. Second guess: AMD. LLY, a defensive health care stock, should have the lowest beta.	MSTR had the highest beta, $\hat{\beta} = 3.54$ (NVDA 2.21, AMD 2.46, META 1.15, LLY 0.45). Prediction supported.

Prediction recorded on: 2026-10-16 00:28 (based only on the Q1 descriptive statistics; no regression was run)

Setup (supplied)

Configuration

Edit **only** this cell. Tickers must match the CSV column headers exactly, and all assets must use the same currency. For your own downloaded data, see `data_guide.pdf` for the settings to change.

```
from pathlib import Path
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt
import portfolio_model as model

config = {
    "stocks": ["LLY", "META", "MSTR", "NVDA", "AMD"],
    "benchmark": "SPY",
    "currency": "USD",
    "prices_file": "data_my_stocks/prices_daily.csv",
    "riskfree_file": "data_my_stocks/riskfree_daily.csv",
    "riskfree_date_kind": "actual quote date",
    "first_return_month": "2021-10",
    "last_return_month": "2026-09",
    "as_of_date": "2026-10-15",
    "preferences_file": "data/certainty_equivalents.csv",
}
stocks = config["stocks"]
benchmark = config["benchmark"]
asset_columns = stocks + [benchmark]
```

Data files

- `data/prices_daily.csv`: a `date` column (`YYYY-MM-DD`), then the five stocks and the benchmark. Use adjusted prices. Leave missing prices blank; never fill them with zero or the previous value.
- `data/riskfree_daily.csv`: `date,annual_yield_percent`, where 2.7% is stored as `2.7`. Each return month needs a quote dated in the month before it. For your own data, use the quote's actual date and set `riskfree_date_kind` to `actual quote date`.
- The supplied Hong Kong files are **practice data**. Their bill dates are period labels, not verified quote dates; say so if you use them.

The cell below aligns the data for you. It uses the last day each month with prices for all six assets, requires at least 60 complete monthly returns ending no later than September 2026, and stops with an error message if a month or bill quote is missing.

```
Common sample: 2021-10 to 2026-09; T=60 monthly returns.
Currency declared by student: USD.
Latest bill-series date: 2026-09-30; annual decimal=0.042000.
Bill date meaning: actual quote date.
R contains raw simple returns; X contains excess returns. Do not subtract cash twice.
```

`R` holds monthly returns and `X` holds monthly **excess** returns. The code checks dates and arithmetic, but you must check that the prices are adjusted and that the bill matches your currency.

1. Choose stocks and calculate returns (15 marks)

Task to Student:

Name	Ticker	Sector	Reason
Eli Lilly	LLY	Health care (pharmaceuticals)	Defensive growth from obesity and diabetes drugs; low link to tech.
Meta Platforms	META	Communication services	Large-cap digital advertising and AI spending.

Name	Ticker	Sector	Reason
Strategy (MicroStrategy)	MSTR	Information technology (software / bitcoin treasury)	Very high-volatility, bitcoin-linked stock.
NVIDIA	NVDA	Information technology (semiconductors)	Leader in AI chips; high growth and risk.
Advanced Micro Devices	AMD	Information technology (semiconductors)	Chip rival to NVIDIA; shows how same-sector stocks co-move.
Benchmark: SPDR S&P 500 ETF	SPY	Broad US market	Tracks the S&P 500; a comparison, not a holding.

All six assets are US-listed and priced in USD.

Task to AI: show each asset's average monthly return, volatility and average excess return. The example below is supplied; `show_table` prints a table in both HTML and PDF, and you can reuse it later.

Asset	Mean return (%)	Volatility (%)	Mean excess return (%)
LLY	3.193	9.094	2.871
META	2.061	12.350	1.739
MSTR	5.825	31.137	5.503
NVDA	5.092	14.459	4.770
AMD	4.739	20.033	4.416
SPY	1.176	4.523	0.854

The chart below is a plotting example only; it is not marked.

```
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(R.index.to_timestamp(how="end"), 100 * R[benchmark], linewidth=1)
ax.axhline(0, color="grey", linewidth=0.7)
ax.set(title=f"{benchmark}: monthly simple returns",
       xlabel="Return month", ylabel="Return (%)")
fig.tight_layout()
plt.show()
```

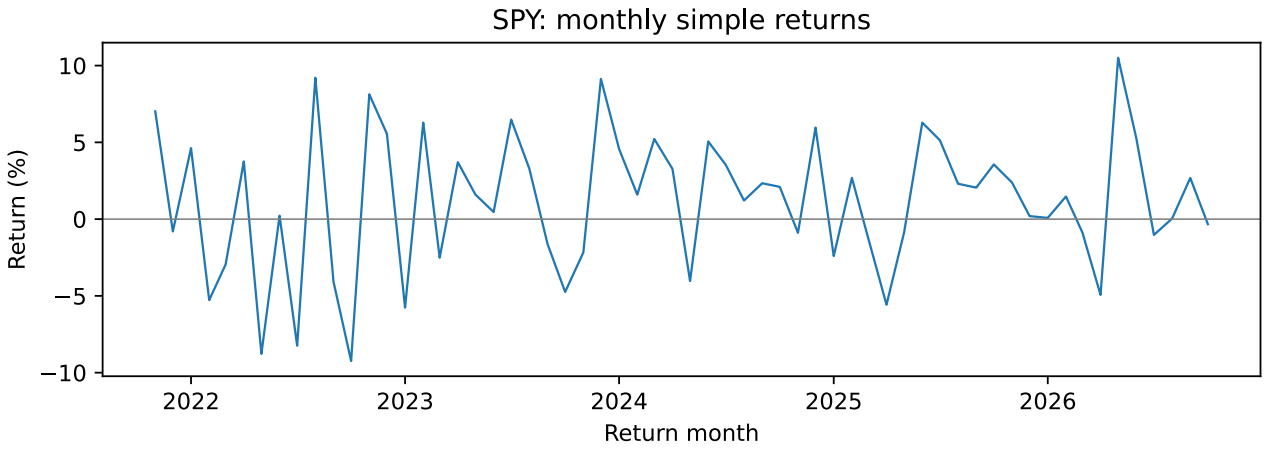


Figure 1: Benchmark monthly simple returns in the aligned sample.

2. Estimate and understand alpha and beta (20 marks)

$$X_{i,t} = \alpha_i + \beta_i X_{M,t} + e_{i,t}, \qquad X_{i,t} = R_{i,t} - R_{f,t}, \quad X_{M,t} = R_{M,t} - R_{f,t}.$$

Task to AI: complete `estimate_index_model`, then switch on the cell below. Add one chosen stock's scatter plot with its fitted SCL, with $X_{M,t}$ on the horizontal axis and $X_{i,t}$ on the vertical axis.

```
estimates, residuals = model.estimate_index_model(X, stocks, benchmark)
show_table(estimates.rename(columns={
    "alpha": r"$\hat\alpha_i$", "beta": r"$\hat\beta_i$",
    "r_squared": r"$R^2$",
}), decimals=4)
```

	$\hat{\alpha}_i$	$\hat{\beta}_i$	R^2
LLY	0.0249	0.4489	0.0493
META	0.0075	1.1545	0.1783
MSTR	0.0248	3.5433	0.2624
NVDA	0.0288	2.2092	0.4744
AMD	0.0231	2.4643	0.3063

```
chosen_stock = "MSTR"
est = estimates.loc[chosen_stock]
x_pct = 100 * X[benchmark]
y_pct = 100 * X[chosen_stock]
grid = np.linspace(x_pct.min(), x_pct.max(), 100)
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(x_pct, y_pct, s=18, alpha=0.7)
ax.plot(grid, 100 * est["alpha"] + est["beta"] * grid, color="red",
        label=f"Fitted SCL: beta = {est['beta']:.2f}, $R^2$ = {est['r_squared']:.2f}")
ax.set(title=f"{chosen_stock} vs {benchmark}: security characteristic line",
       xlabel=f"{benchmark} monthly excess return (%)",
       ylabel=f"{chosen_stock} monthly excess return (%)")
ax.legend()
fig.tight_layout()
plt.show()
```

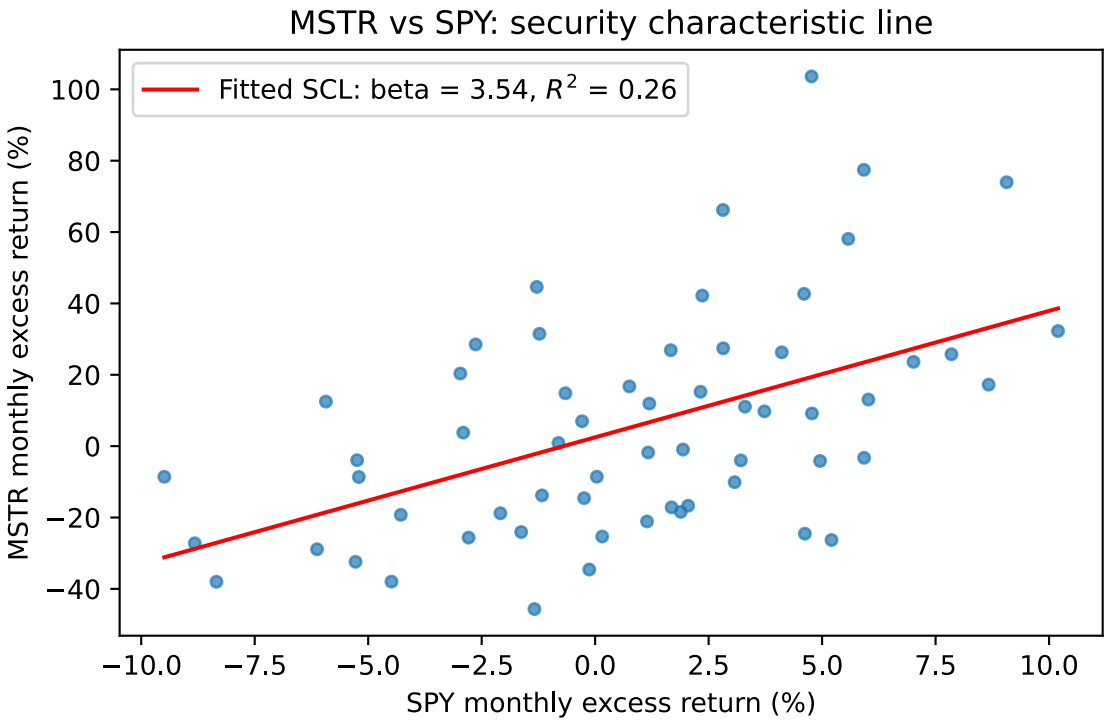


Figure 2: MSTR monthly excess returns against SPY monthly excess returns, with the fitted security characteristic line.

Task to Student: [WRITE: explain the chosen stock's $\hat{\alpha}_i$, $\hat{\beta}_i$ and R^2. If market excess return rises by one percentage point, what does the line predict? Why does the line not predict every monthly return? Was your initial prediction supported?]

3. Understand variance and covariance (20 marks)

$$\sigma_i^2 = \beta_i^2 \sigma_M^2 + \sigma^2(e_i), \qquad \mathbf{Cov}(X_i, X_j) = \beta_i \beta_j \sigma_M^2 \ (i \neq j), \qquad \hat{\sigma}^2(e_i) = \frac{\sum_{t=1}^{T} \hat{e}_{i,t}^2}{T-2}.$$

Task to AI: complete `single_index_covariance`, then switch on the cell below. It shows one 5×5 table of monthly variances (diagonal) and covariances, labelled by ticker.

```
market_variance_m = X[benchmark].var(ddof=1)
covariance_m = model.single_index_covariance(
    estimates["beta"], X[benchmark], residuals
)
show_table(covariance_m, decimals=6)
```

	LLY	META	MSTR	NVDA	AMD
LLY	0.008403	0.001048	0.003217	0.002006	0.002238
META	0.001048	0.015332	0.008275	0.005159	0.005755
MSTR	0.003217	0.008275	0.097994	0.015834	0.017662
NVDA	0.002006	0.005159	0.015834	0.020999	0.011012
AMD	0.002238	0.005755	0.017662	0.011012	0.040578

Task to Student: [WRITE: choose one covariance entry and explain its sign. Why can combining stocks that move differently reduce risk? Give one reason why the zero-residual-covariance assumption might fail for your stocks.]

4. Find and assess a risky portfolio (20 marks)

$$\mathrm{SR}_P = \frac{\mathbb{E}(R_P) - R_f}{\sigma_P}.$$

Task to AI: complete `optimize_risky_portfolio`, then switch on the cell below. Annualize by multiplying monthly mean excess returns and every covariance entry by 12. Show the five weights (nonnegative, summing to 100%), annual excess return, volatility and Sharpe ratio next to the equally weighted portfolio.

This cell also saves your portfolio to `outputs/q5_portfolio.json` for Question 5.

```
annual_excess_means = 12 * X[stocks].mean()
annual_covariance = 12 * covariance_m
portfolio = model.optimize_risky_portfolio(annual_excess_means, annual_covariance)
equal_w = pd.Series(1 / len(stocks), index=stocks)
opt_w = portfolio["weights"].reindex(stocks)
eq_excess = float(equal_w @ annual_excess_means)
eq_vol = float(np.sqrt(equal_w @ annual_covariance.loc[stocks, stocks] @ equal_w))
eq_sharpe = eq_excess / eq_vol
assert (opt_w >= -1e-9).all() and abs(opt_w.sum() - 1) < 1e-9
assert portfolio["sharpe"] >= eq_sharpe - 1e-9
weights_table = pd.DataFrame({
    "Optimized weight (%)": 100 * opt_w,
    "Equal weight (%)": 100 * equal_w,
})
weights_table.index.name = "Stock"
show_table(weights_table, decimals=2)
stats_table = pd.DataFrame({
    "Annual excess return (%)": [100 * portfolio["excess_return"], 100 * eq_excess],
    "Annual volatility (%)": [100 * portfolio["volatility"], 100 * eq_vol],
    "Sharpe ratio": [portfolio["sharpe"], eq_sharpe],
}, index=["Optimized", "Equal weight"])
stats_table.index.name = "Portfolio"
show_table(stats_table, decimals=3)

# Save the exact portfolio and sample used to generate Question 5's menu.
import hashlib
import json
from risk_aversion_quiz import normalize_portfolio_inputs
q5_inputs = normalize_portfolio_inputs({
    "schema_version": 2,
    "weights": portfolio["weights"].to_dict(),
    "annual_premium": portfolio["excess_return"],
    "annual_volatility": portfolio["volatility"],
    "annual_safe_rate": rf_current,
    "currency": config["currency"],
    "safe_rate_date": str(latest_quote_date.date()),
    "safe_rate_date_kind": config["riskfree_date_kind"],
    "first_return_month": str(X.index[0]),
    "last_return_month": str(X.index[-1]),
    "n_months": len(X),
    "data_fingerprint": hashlib.sha256(X.to_csv().encode()).hexdigest(),
})
```

```
Path("outputs").mkdir(exist_ok=True)
Path("outputs/q5_portfolio.json").write_text(
    json.dumps(q5_inputs, indent=2), encoding="utf-8"
)
print("Saved outputs/q5_portfolio.json for Question 5.")
```

Stock	Optimized weight (%)	Equal weight (%)
LLY	54.82	20.00
META	3.22	20.00
MSTR	2.16	20.00
NVDA	31.95	20.00
AMD	7.86	20.00

Portfolio	Annual excess return (%)	Annual volatility (%)	Sharpe ratio
Optimized	43.434	28.914	1.502
Equal weight	46.317	39.662	1.168

```
Saved outputs/q5_portfolio.json for Question 5.
```

Task to Student: [WRITE: check the weights are nonnegative and sum to 100%, and that the Sharpe ratio is at least as high as equal weighting. What do you notice about the holdings? Why can historical results not guarantee future performance?]

Now render the report once. This creates `outputs/q5_portfolio.json`. If $\delta \leq 0$, keep it and explain it in Question 6; do not replace it with a made-up positive return.

5. Work out your risk aversion (10 marks)

P is your Question 4 portfolio, with annual excess return δ, variance σ_P^2 and mean $\mu_P = R_f + \delta$. For $\delta > 0$, let $G = \delta/\sigma_P^2$. The quiz offers five risk-aversion levels $\gamma_m = mG$ (normally $m = 1.25, 2, 3, 4, 5$) and shows each as a certainty equivalent wealth:

$$c_m = \mu_P - \frac{\gamma_m \sigma_P^2}{2}, \qquad W_m = 10000(1 + c_m).$$

Task to AI: run the quiz in a terminal (not inside this file):

```
python risk_aversion_quiz.py --portfolio outputs/q5_portfolio.json
```

It shows P's mean and volatility, its lowest 10%, middle 80% and highest 10% return/wealth ranges, and the menu of W_m labelled A to E. No option is below the lowest-10% outcome, so for a high-Sharpe portfolio the quiz uses a narrower range of m. After the student types a letter, it prints the implied γ_m and saves the choice to `data/certainty_equivalents.csv`. Then complete `infer_risk_aversion` and switch on the cell below, which reads that file.

```
from risk_aversion_quiz import load_preferences
preference_trials = None
gamma = None
```

```
if portfolio["excess_return"] > 0:
    preference_trials = load_preferences(config["preferences_file"], q5_inputs)
    preferences = model.infer_risk_aversion(preference_trials)
    gamma = preferences["gamma"]
    preference_estimates = preferences["estimates"]
    preference_table = pd.DataFrame({
        "Choice": preference_estimates["choice"],
        "CE wealth": preference_estimates["certainty_equivalent_wealth"],
        "CE (%)": 100 * preference_estimates["certainty_equivalent"],
        "Gamma": preference_estimates["gamma"],
    }).set_index("Choice")
    show_table(preference_table, decimals=4)
    print(f"Implied gamma for my selected portfolio: {gamma:.6f}")
else:
    print("Nonpositive expected excess return: numeric gamma is not elicited.")
```

If you change your stocks, data or Question 4 portfolio, run the quiz again: the cell rejects a choice made for a different portfolio.

Task to Student: [WRITE: which letter you chose and why, and what a larger gamma means. This is a coarse estimate from a five-option menu; say if none of the options fit you well. If $\delta \leq 0$, skip the quiz and explain why in Question 6.]

6. Choose your stock-and-safe-asset mix (15 marks)

The **final portfolio** F puts a share w_r^* of wealth in P (keeping its Question 4 weights w_i) and $1 - w_r^*$ in the safe asset, using your Question 5 γ:

$$w_r^* = \frac{\delta}{\gamma \sigma_P^2}, \qquad \mathbb{E}(R_F) = R_f + w_r^* \delta, \qquad \sigma_F = w_r^* \sigma_P.$$

Task to AI: complete `allocate_portfolio`, then switch on the cell below. For wealth of 10,000, show w_r^*, each stock's percentage $w_r^* w_i$ and amount $10000\, w_r^* w_i$, the safe asset's percentage $1 - w_r^*$ and amount, and $\mathbb{E}(R_F)$ and σ_F. Do not clip w_r^*.

```
allocation_premium = portfolio["excess_return"]
allocation_volatility = portfolio["volatility"]
mix = model.allocate_portfolio(
    portfolio["weights"], allocation_premium, allocation_volatility,
    gamma, rf_current, 10000
)
if allocation_premium > 0:
    assert 0 < mix["risky_share"] < 1
else:
    assert mix["risky_share"] == 0
# TODO: display the risky share, holdings and annual mix statistics.
```

Task to Student: [WRITE your prediction **before** running the next cell: if P's expected excess return δ were halved, with weights, variance and γ unchanged, what would happen to w_r^*? Why?]

Task to AI: switch on the cell below and compare the original and new w_r^*.

```
half_mix = model.allocate_portfolio(
    portfolio["weights"], 0.5 * allocation_premium,
    allocation_volatility, gamma, rf_current, 10000
)
# TODO: display and compare the original and new risky shares.
```

Nonpositive-premium exception: if $\delta \leq 0$, report 100% in the safe asset, expected return R_f and zero volatility; the half-premium test also gives 0%. Explain why positive risk aversion cannot give an interior allocation.

Task to Student: [WRITE: how does larger risk aversion change w_r^*? Compare your prediction with the half-premium result. In at most 100 words, would you accept this allocation based on these estimates?]

(b) AI evidence entry (for Q6)

Use the half-premium test from Question 6.

Item	My entry
AI tool and relevant claim	[NAME the tool and QUOTE its claim]
My prediction before testing	[WRITE the prediction and reason]
My test and result	[STATE the fixed inputs and original/new risky shares]
My decision	[EXPLAIN whether I accept or revise the recommendation]

(c) Data sources (for Q1)

- **Prices:** Yahoo Finance via `yfinance` (`tools/download_data.py`), daily prices adjusted for splits and dividends.
- **Assets:** LLY, META, MSTR, NVDA, AMD; benchmark SPY; all in USD.
- **Sample:** 60 monthly returns, October 2021 to September 2026, from the last common trading day of each month (the starting price is from September 2021).
- **Safe rate:** FRED 3-month Treasury yield (DGS3MO), using each month's return with the previous month's last quote, dated by actual quote date. Monthly rate is the annual percent divided by 1200.
- I did **not** use the supplied HK practice data; the data is in `data_my_stocks/`.

Render and submit

From the folder containing this file, run (Typst produces the PDF; no LaTeX needed):

```
quarto render studentID_assignment_one.qmd --to typst
```

Open the PDF and check that every table, figure and equation appears and no placeholder is left. Submit **three files**: `studentID_assignment_one.qmd`, `studentID_assignment_one.pdf` and your completed `portfolio_model.py`.